



Цифровые технологии управления
транспортными процессами

Сервис-ориентированное
программирование



GraphQL



GraphQL

- GraphQL - язык запросов для API и серверная среда для выполнения этих запросов с использованием системы типов.
- Создатель
 - Facebook (Meta)*, 2012, внутренний проект, опенсорс с 2015 года
- Текущий владелец
 - GraphQL Foundation (под Linux Foundation)
- Спецификация
 - spec.graphql.org - последняя редакция сентябрь 2025
- Назначение
 - Единый гибкий endpoint для клиентских запросов к данным



Проблематика избыточных данных

- Представьте, что вам нужно отобразить список записей (posts), и под каждым опубликовать список лайков (likes), включая имена пользователей и аватары.
- *REST*:
- Проблема - клиент получает 15+ полей пользователя, хотя нужны только name и avatar. На мобильных устройствах это критично - лишний трафик, медленнее парсинг, выше энергопотребление.

```
# возвращает ВСЕ поля поста
GET /api/posts
# возвращает лайки с ПОЛНЫМИ объектами
пользователей
GET /api/posts/1/likes
```



Проблема недостатка данных

- Для мобильного приложения мы выяснили, что лайки замедляют работу.
- *Создаём два endpoint:*

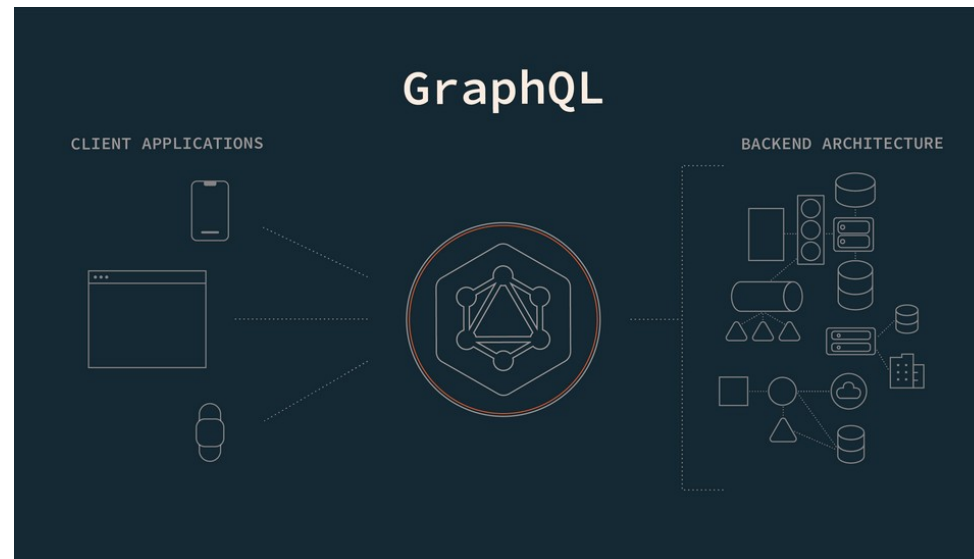
```
GET /api/posts - посты без лайков  
GET /api/posts?include=likes - посты с лайками
```

- Но теперь нужны и комментарии, и автор поста, и рекомендации - количество endpoint растёт экспоненциально.
- При этом:
 - Посты хранятся в PostgreSQL
 - Лайки - в Redis
 - Рекомендации - в ML-сервисе
 - Комментарии - в MongoDB
- REST решение про множество endpoint + множество запросов от клиента = $N+1$ проблема на уровне HTTP



Решение

- Facebook придумал концептуально простое решение: вместо того, чтобы иметь множество «глупых» endpoint, лучше иметь один «умный» endpoint, который будет способен работать со сложными запросами и придавать данным такую форму, какую запрашивает клиент.
- Клиент отправляет один запрос, описывающий полную структуру нужных данных, и получает один ответ с данными из всех источников





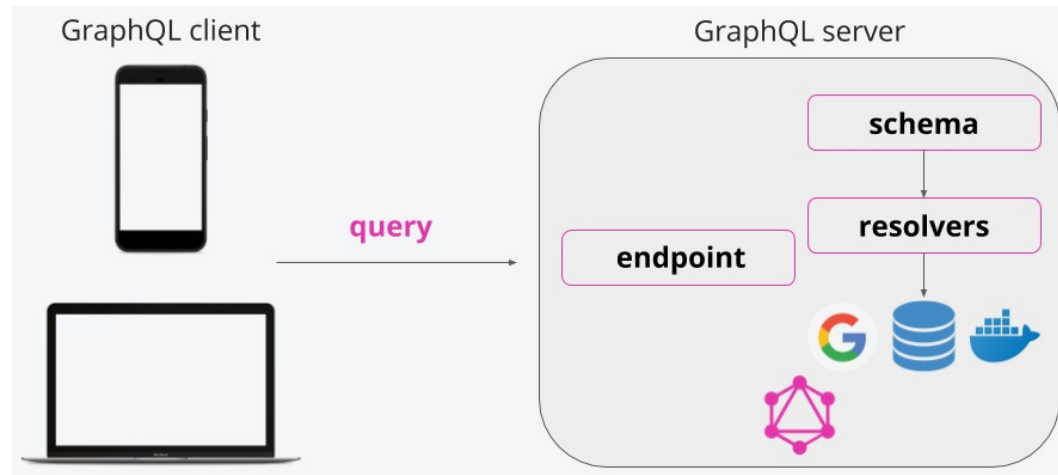
Архитектура GraphQL

- Schema (Схема) - контракт: какие типы данных доступны и какие операции ВОЗМОЖНЫ
- Resolvers (Резолверы) - функции, которые «знают», откуда взять данные для каждого поля
- Data Sources - любые источники: SQL/NoSQL БД, REST API, gRPC, файлы, кеш



Schema

В центре любой имплементации GraphQL API лежит схема данных — это описание того, с какими типами данных он может работать и какие типы данных может вернуть в ответ на запрос (они описаны в системе типов GraphQL). Проще говоря, для работы с любым API пользователю необходимо знать, какие типы объектов можно получить, какие поля выбрать, какие поля доступны во внутренних объектах и т. д. Именно в этом нам поможет схема.





GraphQL Schema Language (SDL)

- Универсальный язык описания схемы, не зависящий от языка программирования
- *Скалярные типы:*

Тип	Описание	Пример
Int	32-битное целое	age: Int
Float	Число с плавающей точкой	price: Float
String	Строка UTF-8	name: String
Boolean	true / false	active: Boolean
ID	Уникальный идентификатор (сериализуется как строка)	id: ID



GraphQL Schema Language (SDL)

- Универсальный язык описания схемы, не зависящий от языка программирования
- *Модификаторы:*
 - String! - non-nullable (обязательное поле)
 - [String] - список
 - [String!]! - обязательный список обязательных строк



GraphQL Schema Language (SDL). Пользовательские типы

```
type User {  
  id: ID!  
  name: String!  
  email: String!  
  age: Int  
  posts: [Post!]!      # связь с другим  
  createdAt: DateTime  # custom scalar  
}  
  
type Post {  
  id: ID!  
  title: String!  
  content: String!  
  author: User!         # обратная связь  
  likes: [User!]!  
  comments: [Comment!]!  
}
```



Input Types и Enum

- Input type - для передачи структурированных аргументов

```
input CreateUserInput
{
  name: String!
  email: String!
  age: Int
}
```

```
input UserFilter {
  nameContains: String
  minAge: Int
  maxAge: Int
}
```



Input Types и Enum

- Enum - фиксированный набор значений

```
enum Role {  
  ADMIN  
  MODERATOR  
  USER  
}  
  
enum SortOrder  
{  
  ASC  
  DESC  
}
```



Interface и Union

- Interface и Union - для полиморфизма

```
interface Node {  
  id: ID!  
}
```

```
union SearchResult = User | Post |  
Comment
```



Язык запросов GraphQL

- Язык запросов GraphQL оперирует тремя основными концепциями:
 - queries, запросы - с их помощью получают данные с сервера.
 - mutations, изменения - модификация данных на сервере и их обновление.
 - subscriptions, подписки - методы поддержания постоянной связи с сервером.



Queries. Определение в схеме

```
type Query {  
  users: [User!]!  
  user(id: ID!): User  
  posts(filter: PostFilter):  
  [Post!]!  
}
```



Queries. Запрос клиента

```
query GetUserWithPosts
{
  user(id: "42") {
    name
    email
    posts {
      title
      likes {
        name
      }
    }
  }
}
```




Queries. Ответ сервера (JSON)

```
{
  "data": {
    "user": {
      "name": "Иван Петров",
      "email": "ivan@example.com",
      "posts": [
        {
          "title": "Введение в GraphQL",
          "likes": [{ "name": "Анна" }, { "name":
"Мария" }]
        }
      ]
    }
  }
}
```



Аргумент

- Представляют собой набор пар «ключ - значение», которые привязаны к определенному полю. Они передаются на сервер и влияют на то, как будут получены данные для определенного поля. Аргументы могут быть и литералами, и переменными. Их можно применять на любых полях вне зависимости от уровня их вложенности. Они обязательно должны быть именованными, а также могут быть обязательными или опциональными (если аргументы опциональные, то их значение должно быть задано по умолчанию).
- Например, в примере к полю User привязан аргумент id (в данном примере литерал типа ID)



Аргумент

```
query {  
  user(id: 1) {  
    name  
    posts(first: 5, orderBy: { field: CREATED_AT, direction:  
DESC }) {  
      title  
    }  
  }  
}
```



Аргумент и переменные

```
query GetUser($userId: ID!, $postsLimit: Int =  
10) {  
  user(id: $userId) {  
    name  
    posts(first: $postsLimit) {  
      title  
    }  
  }  
}
```

Переменные передаются отдельно
(обычно JSON)

```
{  
  "userId": "42",  
  "postsLimit": 5  
}
```



mutations

- Специальный тип, который, в отличие от `query`, используется для изменения данных. Этот процесс можно рассматривать как аналогичный концепции CUD (Create, Update, Delete) в стандарте REST. Да, конечно, это рекомендация, и на практике не запрещено использовать для изменения данных и `query`, но все же так делать не стоит.
- Ведь основное отличие `query` от `mutation` — их поведение при получении результатов запросов: при выполнении `query` обработка полей результата будет выполняться параллельно, так как они не изменяют данные. В свою очередь, в `mutations` это выполняется последовательно, чтобы обеспечить целостность данных.



Mutations. Определение в схеме

```
type Mutation {  
  createUser(input: CreateUserInput!): User!  
  updateUser(id: ID!, input: UpdateUserInput!):  
  User!  
  deleteUser(id: ID!): Boolean!  
}
```



Mutations. Запрос

```
mutation CreateNewUser($input:
CreateUserInput!) {
  createUser(input: $input) {
    id
    name
    email
  }
}
```



Subscriptions

- Специальный тип данных, который позволяет реализовать систему publisher/subscriber.
- То есть это способ отправки данных клиенту, который «слушает» эти данные в реальном времени. В отличие от предыдущих 2 специальных типов, тут ответ сервера приходит не единоразово, а каждый раз при срабатывании определенных условий на сервере при условии, что клиент «слушает» эти данные. Можно сказать, что это аналог push-нотификаций, которые работают в одной системе со всеми другими типами запросов - query и mutations.
- Большой плюс данного подхода - то, что клиент и в этом случае может с помощью аргументов определить, при каком условии результат с сервера должен приходиться на клиент.



Subscriptions. Определение

```
type Subscription {  
  postCreated: Post!  
  userOnline(userId: ID!):  
  UserStatus!  
  commentAdded(postId: ID!):  
  Comment!  
}
```

- При обновлении данных сервер выполняет определённый в подписке запрос GraphQL и рассылает обновлённые данные клиентам.



Subscriptions. Использование

```
subscription OnNewPost {  
  postCreated {  
    id  
    title  
    author {  
      name  
    }  
  }  
}
```

- При обновлении данных сервер выполняет определённый в подписке запрос GraphQL и рассылает обновлённые данные клиентам.
- Используются WebSocket и SSE (Server-Sent Events)



Fragments и директивы

- Fragments - именованные наборы полей для DRY

```
fragment UserBasicInfo on User
{
  id
  name
  email
  avatar
}
```

```
query {
  user(id: 1) {
    ...UserBasicInfo
    posts { title }
  }
  admins {
    ...UserBasicInfo
    role
  }
}
```



Директивы - условное включение полей

- Fragments - именованные наборы полей для DRY

```
query GetPosts($withComments: Boolean!)  
{  
  posts {  
    title  
    comments @include(if:  
$withComments) {  
      text  
      author { name }  
    }  
  }  
}
```

Директива	Описание
@include(if: Boolean!)	Включить поле, если true
@skip(if: Boolean!)	Пропустить поле, если true
@deprecated(reason: String)	Пометить поле как устаревшее
@defer (2024+)	Отложенная доставка (incremental delivery)
@stream (2024+)	Потоковая доставка списков



@defer и @stream

- Инкрементальная доставка данных
 - Проблема - запрос возвращает и быстрые, и медленные данные. Клиент ждёт самого медленного поля
 - Решение с @defer:

@stream - аналогично, но для списков (элементы приходят по мере загрузки)

```
query GetDashboard {  
  user(id: 1) {  
    name           # быстро — из кеша  
    email          # быстро  
  
    analytics @defer { # медленно — тяжёлый  
      расчёт  
      totalViews  
      revenue  
      conversionRate  
    }  
  }  
}
```



Когда использовать GraphQL?

- Фронтенд требует гибких структур данных
- Мобильные приложения с ограниченным каналом
- Агрегация данных из нескольких микросервисов (BFF-паттерн)
- Сложные, глубоко связанные модели данных
- Нужна быстрая итерация фронтенда



Когда использовать REST?

- Простые CRUD-операции с предсказуемой структурой
- Публичные API для сторонних разработчиков
- HTTP-кеширование критично (CDN, прокси)
- Команда имеет большой опыт с REST



Безопасность GraphQL

- GraphQL по умолчанию более уязвим, чем REST, из-за гибкости запросов. Необходимы явные защитные меры:
 - Query Depth Limiting - ограничение глубины запросов

```
# Атака: бесконечная рекурсия
query {
  user(id: 1) {
    posts {
      author {
        posts {
          author {
            posts {
              ... } # глубина ∞, DoS-
            }
          }
        }
      }
    }
  }
}
```



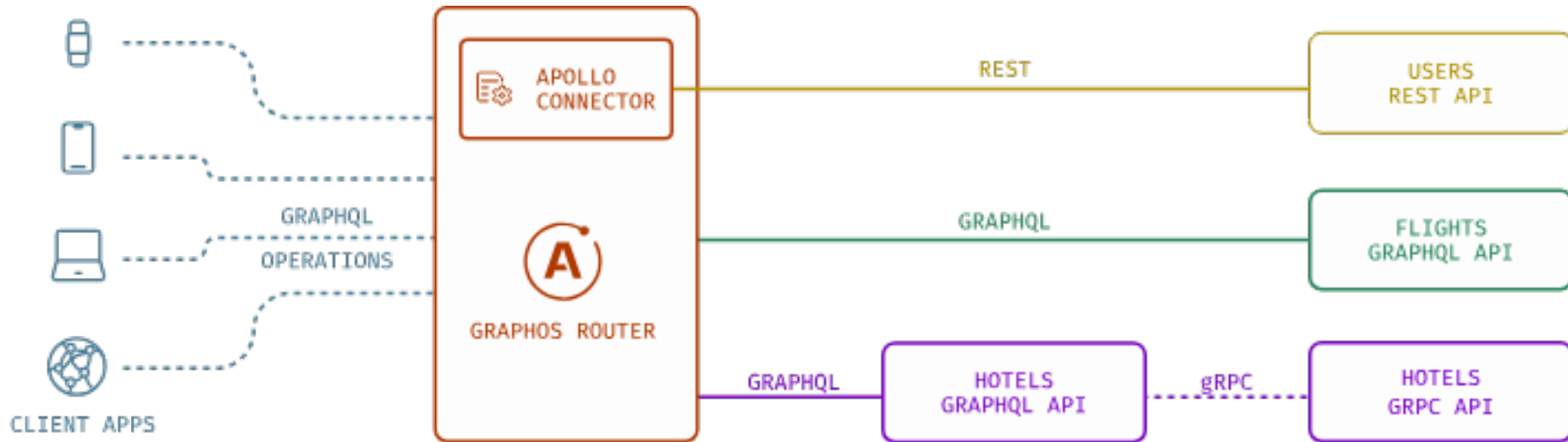

Безопасность GraphQL

- GraphQL по умолчанию более уязвим, чем REST, из-за гибкости запросов. Необходимы явные защитные меры:
 - Query Cost Analysis - анализ стоимости запроса:
 - Каждому полю назначается «вес» (cost)
 - Перед выполнением вычисляется общая стоимость
 - Запросы с $\text{cost} > \text{threshold}$ отклоняются
 - Persisted Queries (сохранённые запросы):
 - Клиент отправляет не текст запроса, а хэш
 - Сервер выполняет только запросы из «белого списка»
 - Полностью блокирует произвольные запросы



Federation

- В крупных организациях один GraphQL-сервер становится узким местом. Federation - архитектурный паттерн для распределённого GraphQL





Будут вопросы? Не стесняйтесь!

